

Brain Tumor MRI Classification

End-to-End Deep Learning Application Using PyTorch, FastAPI, React, and AWS

Project Category: Deep Learning and Computer Vision

Algorithm: Custom Convolutional Neural Network

Prepared By: Rajasekar S

Role: Project Engineer

Implementation: Local and AWS End-to-End Deployment

Document Version: 1.0

Date: 20-09-2026

Disclaimer

This project was developed exclusively for educational, learning, and portfolio demonstration purposes.

The model has not undergone clinical validation, external hospital validation, medical review, security certification, or regulatory approval. It must not be used for medical diagnosis, treatment recommendations, clinical decision-making, or any healthcare-related purpose.

The model output represents a prediction produced from the training dataset and must not be interpreted as medical advice.

1. Executive Summary

This project implements an end-to-end deep learning application for classifying brain MRI images into four categories:

- Glioma
- Meningioma
- No Tumor
- Pituitary Tumor

A custom Convolutional Neural Network was developed using PyTorch. The model was trained using labelled brain MRI images and evaluated using a held-out test dataset. Data augmentation,

weighted loss, learning-rate scheduling, early stopping, checkpointing, and hyperparameter tuning were incorporated into the training workflow.

Following model development, the notebook implementation was converted into a modular production-style Python project. The selected PyTorch model was exposed through a FastAPI backend that supports image upload, input validation, model inference, class probability generation, and health monitoring.

A React frontend was developed using Vite. The frontend allows users to upload an MRI image, preview the selected image, submit it for prediction, and view the predicted category, confidence score, probabilities for all four classes, and manual-review status.

The complete application was first tested locally. It was then extended to AWS:

- The React production build was hosted as a static website on Amazon S3.
- The trained model artifact was stored in a private Amazon S3 bucket.
- The FastAPI backend was deployed on an Amazon EC2 instance.
- An EC2 IAM role was used to securely retrieve the model checkpoint.
- Nginx was configured as a reverse proxy.
- systemd was configured to run the FastAPI service continuously.
- Cross-Origin Resource Sharing was configured to permit communication between the S3-hosted frontend and the EC2 backend.

The project demonstrates practical knowledge in deep learning, computer vision, Python software engineering, REST API development, frontend integration, AWS deployment, model artifact management, and end-to-end machine learning application delivery.

2. Project Objective

The primary objective of this project was to demonstrate practical deep learning and machine learning engineering skills by developing a complete solution rather than limiting the work to model training in a notebook.

The main objectives were:

1. Organize and validate a multiclass image dataset.
2. Create reproducible training, validation, and test workflows.
3. Develop a custom CNN using PyTorch.
4. Apply image preprocessing and data augmentation.
5. Handle class imbalance using a weighted loss function.
6. Train the model using Apple M1 MPS acceleration.
7. Evaluate the model using multiple classification metrics.

8. Perform controlled hyperparameter tuning.
 9. Save and restore reusable model checkpoints.
 10. Convert notebook code into production-style modules.
 11. Expose the trained model through FastAPI.
 12. Build a React-based image-upload interface.
 13. Complete local frontend-to-backend integration.
 14. Deploy the frontend using Amazon S3 static website hosting.
 15. Deploy the backend using Amazon EC2.
 16. Store and manage model artifacts using a private Amazon S3 bucket.
 17. Document the full development and deployment lifecycle.
-

3. Business and Learning Context

The project was selected because medical-image classification provides a practical and easily understandable demonstration of deep learning.

Common learning datasets such as MNIST and CIFAR are useful for understanding CNN fundamentals. However, a brain MRI classification problem offers a more realistic use case and provides stronger portfolio evidence.

The project demonstrates the ability to work across multiple layers:

- Dataset preparation
- Deep learning model development
- Model evaluation
- Production code organization
- API development
- Frontend development
- Local integration
- AWS infrastructure
- Deployment validation
- Technical documentation

The project does not present the model as a clinical solution. Its purpose is to demonstrate technical implementation and end-to-end ML engineering capability.

4. Problem Statement

Manual analysis of medical images requires specialized knowledge, careful examination, and reliable clinical procedures.

For this educational project, the objective was to train a CNN to learn visual patterns from labelled brain MRI images and classify an uploaded image into one of four predefined categories.

The classification categories are:

1. Glioma
2. Meningioma
3. No Tumor
4. Pituitary Tumor

The application accepts one MRI image and returns:

- Predicted class
- Prediction confidence
- Probability for each class
- Manual-review recommendation based on a confidence threshold
- Educational-use disclaimer

The model does not perform:

- Tumor segmentation
- Tumor localization
- Tumor-size estimation
- Clinical report generation
- Patient-risk assessment
- Treatment recommendation

5. Technology Stack

5.1 Programming and Machine Learning

- Python
- PyTorch
- Torchvision
- NumPy
- Pandas
- Scikit-learn
- Pillow

- TQDM

5.2 Data Visualization

- Matplotlib
- Seaborn

5.3 Backend

- FastAPI
- Uvicorn
- Pydantic
- Python Multipart
- Boto3

5.4 Frontend

- React
- Vite
- JavaScript
- CSS
- Lucide React

5.5 Local Development Tools

- Apple M1 Mac
- PyTorch MPS
- Visual Studio Code
- Jupyter Notebook
- Python virtual environment
- Postman
- Git
- GitHub

5.6 AWS Services

- Amazon S3 for React static website hosting
- Amazon S3 for private model artifact storage
- Amazon EC2 for FastAPI backend hosting
- AWS IAM role for secure model access
- EC2 security group
- Elastic IP
- Nginx reverse proxy
- systemd service management

6. Dataset Information

6.1 Dataset Overview

The dataset contains labelled brain MRI images organized into four categories:

- Glioma
- Meningioma
- No Tumor
- Pituitary Tumor

The dataset includes separate training and testing directories.

Dataset Name: Brain Tumor MRI Dataset

Dataset Source: Mendeley Data

Dataset Link: [Insert Dataset URL]

License: Creative Commons Attribution 4.0

Image Type: Brain MRI images

Number of Classes: Four

6.2 Dataset Structure

```
data/
├── Epic and CSCR hospital Dataset/
├── train/
│   ├── glioma/
│   ├── meningioma/
│   ├── notumor/
│   └── pituitary/
├── test/
├── glioma/
├── meningioma/
├── notumor/
└── pituitary/
```

The test folder supplied by the dataset was kept separate from model training and hyperparameter selection.

The supplied training directory was divided into:

Training subset: 85%

Validation subset: 15%

6.3 Dataset Screenshots

Figure 1: Dataset folder structure

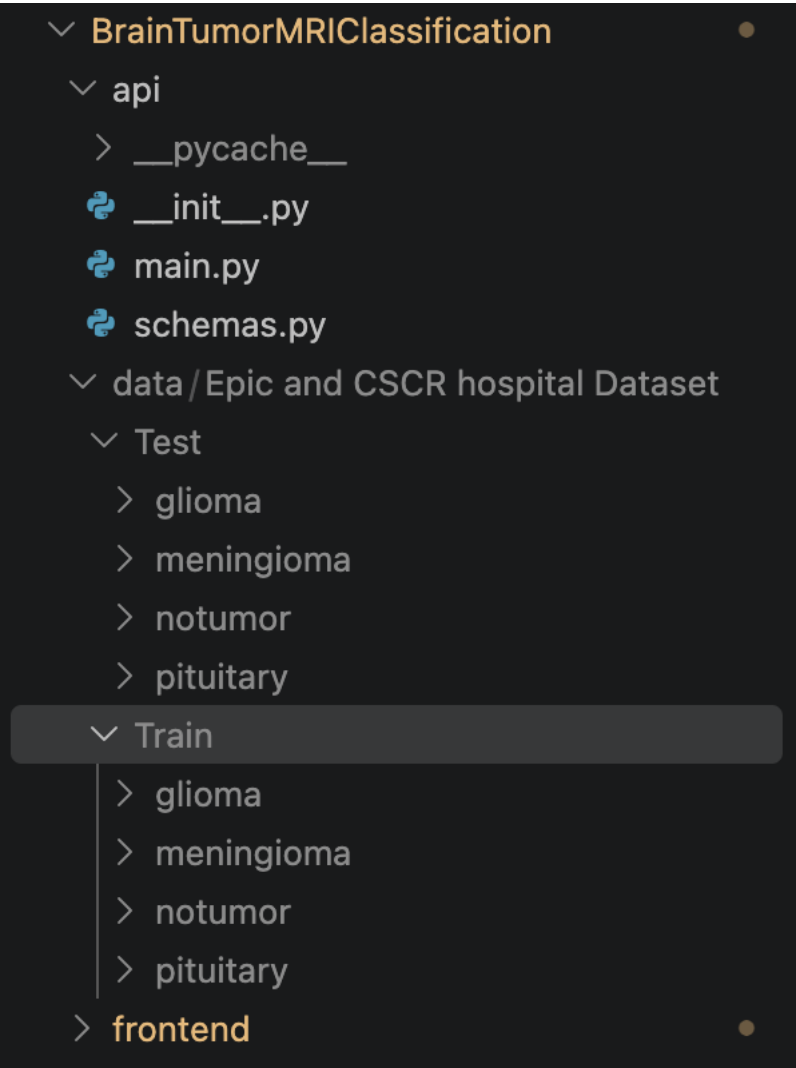


Figure 2: Training and testing class distribution

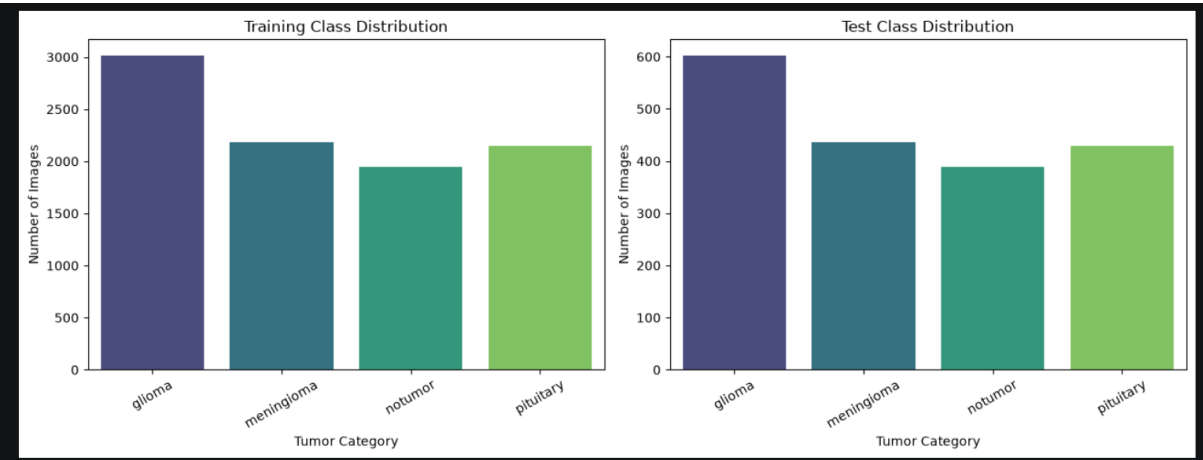
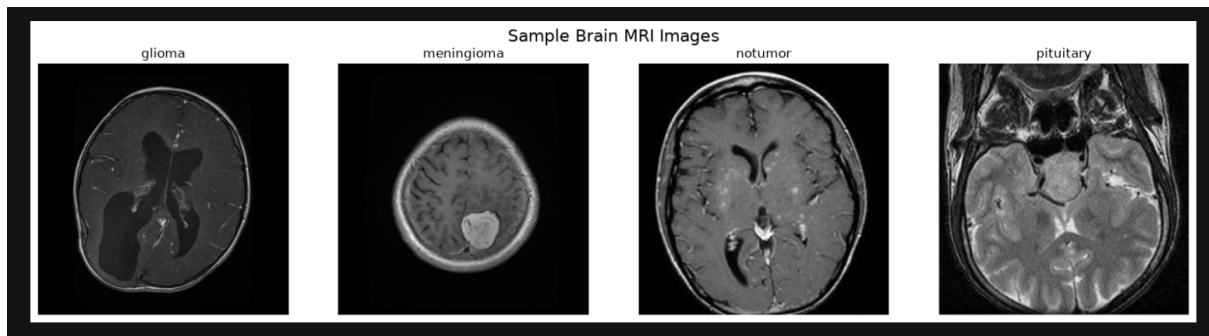


Figure 3: Sample images from all classes



7. Dataset Validation

The dataset was validated before training.

The following checks were performed:

- Confirmed that the training directory exists.
- Confirmed that the testing directory exists.
- Verified that both directories contain the same four classes.
- Checked for missing values in the generated image index.
- Checked for duplicate image paths.
- Ignored hidden files and unsupported formats.
- Verified supported image extensions.
- Converted every image to RGB.
- Added exception handling for corrupted or unreadable images.
- Verified the DataLoader output dimensions.

8. Training, Validation, and Test Strategy

The dataset usage was clearly separated.

Provided train directory

- └ Training subset: 85%
- └ Validation subset: 15%

Provided test directory

- └ Final evaluation: 100%

8.1 Training Subset

The training subset was used for:

- Model weight updates
- Data augmentation
- Loss calculation
- Accuracy calculation
- Class-weight calculation
- Optimizer updates

8.2 Validation Subset

The validation subset was used for:

- Validation loss
- Validation accuracy
- Validation macro F1
- Model checkpoint selection
- Learning-rate scheduling
- Early stopping
- Hyperparameter comparison

8.3 Test Dataset

The test dataset was reserved for final evaluation.

It was used to calculate:

- Test loss
- Test accuracy
- Macro precision
- Macro recall
- Macro F1
- Weighted F1
- Per-class precision
- Per-class recall
- Per-class F1
- Confusion matrix
- Misclassification results
- Prediction confidence

The test dataset was not used for model weight updates.

9. Image Preprocessing

The following preprocessing was applied:

1. Open the image using Pillow.
2. Convert the image to RGB.
3. Resize the image to 224×224 .
4. Convert the image to a PyTorch tensor.
5. Normalize the tensor.

The normalization values were:

Mean: [0.5, 0.5, 0.5]

Standard deviation: [0.5, 0.5, 0.5]

This normalization transformed the image tensors into a range suitable for CNN training.

10. Data Augmentation

Data augmentation was applied only to the training images.

The augmentations included:

- Random horizontal flip
- Random rotation up to 10 degrees
- Random affine transformation
- Minor image translation
- Minor image scaling
- Brightness adjustment
- Contrast adjustment

The validation, test, and inference images received deterministic preprocessing only.

The purpose of augmentation was to expose the model to controlled variations and reduce overfitting.

11. Custom PyTorch Dataset

A custom PyTorch Dataset class was implemented.

Its responsibilities included:

- Storing the image DataFrame
- Loading each MRI image

- Converting the image to RGB
- Applying the appropriate transformation
- Converting the class name into an integer label
- Returning the image tensor and class label
- Optionally returning the image path for evaluation

Three DataLoaders were created:

- Training DataLoader
- Validation DataLoader
- Test DataLoader

The training DataLoader used shuffling. The validation and test DataLoaders used a fixed order.

12. macOS DataLoader Issue and Resolution

When multiple DataLoader workers were used inside Jupyter on macOS, the custom Dataset class produced a multiprocessing error.

The error indicated that the worker process could not retrieve the `BrainTumorDataset` class.

The issue was caused by:

- macOS multiprocessing spawn behaviour
- Custom class definition inside an interactive notebook
- Serialization of the Dataset instance

The issue was resolved by using: `num_workers = 0`

This caused image loading to occur in the main Python process.

The CNN training still used Apple M1 MPS acceleration.

13. CNN Model Architecture

A custom four-block CNN was implemented using PyTorch.

13.1 Convolution Block 1

Input channels: 3

Output channels: 32

Kernel size: 3×3

Padding: 1

Batch normalization

ReLU activation

Max pooling

13.2 Convolution Block 2

Input channels: 32

Output channels: 64

Kernel size: 3×3

Padding: 1

Batch normalization

ReLU activation

Max pooling

13.3 Convolution Block 3

Input channels: 64

Output channels: 128

Kernel size: 3×3

Padding: 1

Batch normalization

ReLU activation

Max pooling

13.4 Convolution Block 4

Input channels: 128

Output channels: 256

Kernel size: 3×3

Padding: 1

Batch normalization

ReLU activation

Max pooling

13.5 Classifier

The convolutional layers were followed by:

Adaptive average pooling

Flatten

Linear layer: 256 to 128

ReLU

Dropout: 0.40

Linear layer: 128 to 4

The model returned four raw logits.

A Softmax layer was not included inside the model during training because `CrossEntropyLoss` expects raw logits.

14. Training Configuration

The final training configuration was:

Image size: 224×224
Batch size: 32
Maximum epochs: 20
Optimizer: AdamW
Learning rate: 0.001
Weight decay: 0.0001
Dropout: 0.40
Loss function: Weighted CrossEntropyLoss
Scheduler: ReduceLROnPlateau
Early-stopping patience: 5
Random seed: 42
Workers: 0
Training device: Apple M1 MPS

15. Class Imbalance Handling

The number of images varied between classes.

Class weights were calculated using only the training subset.

The calculated weights were passed to `CrossEntropyLoss`.

This increased the loss contribution of smaller classes and reduced the possibility that the model would optimise primarily for the largest category.

16. Optimizer and Scheduler

AdamW was used as the optimizer.

AdamW was selected because it provides adaptive learning rates and handles weight decay separately from the gradient update.

`ReduceLROnPlateau` was used as the learning-rate scheduler.

The scheduler reduced the learning rate when validation performance stopped improving.

This allowed the model to make smaller parameter updates during later training stages.

17. Early Stopping and Checkpointing

Early stopping was configured with a patience value of five epochs.

When validation performance improved:

- The model weights were copied.
- The best epoch was recorded.
- The best validation metrics were recorded.
- The checkpoint was saved.

The checkpoint contained:

- Model state dictionary
- Class-to-index mapping
- Input image size
- Learning rate
- Weight decay
- Dropout value
- Best epoch
- Best validation loss
- Best validation macro F1

This allowed the best model to be reused without retraining.

18. Local Training Environment

The model was trained on an Apple M1 Mac using PyTorch MPS acceleration.

The device-selection logic used:

1. MPS, if available
2. CUDA, if available
3. CPU as fallback

The 20-epoch training run required approximately 53 minutes.

The average runtime was approximately: 2.65 minutes per epoch

The training time included:

- Image loading
- Image augmentation
- Forward propagation
- Backpropagation
- Weight updates
- Validation
- Metric calculation

Figure : Final training epochs

```
Epoch 20/20 | Train Loss: 0.3056 | Validation Loss: 0.3940 | Train Accuracy: 0.8850 | Validation Accuracy:  
Epochs without improvement: 4
```

```
Training time in minutes: 81.86
```

```
Best validation loss: 0.2897
```

```
Saved model: /Users/ra20549220/Desktop/ML/BrainTumorMRIClassification/outputs/best\_brain\_tumor\_cnn.pth
```

19. Training Curves

Training and validation loss were monitored after every epoch.

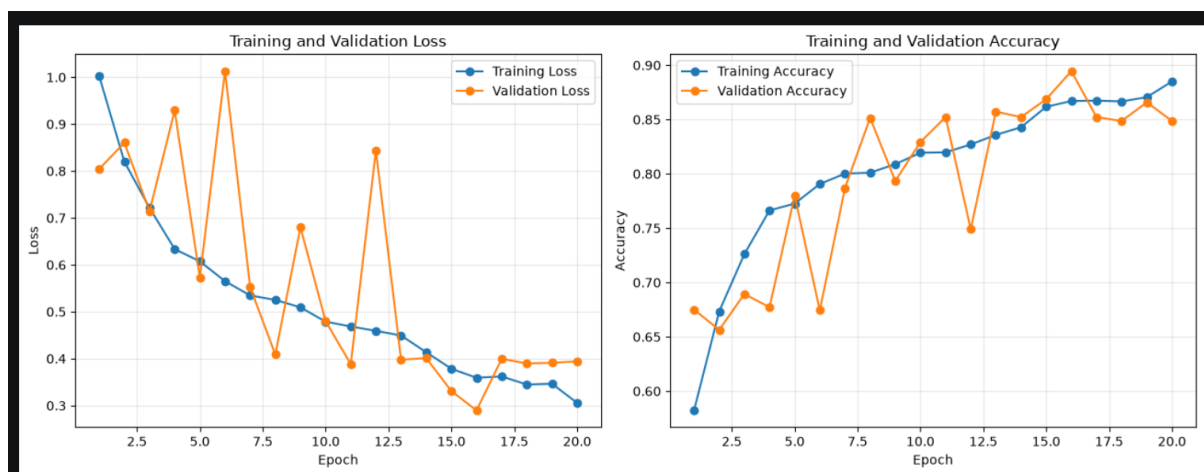
Training and validation accuracy were also recorded.

The curves were analysed to identify:

- Overfitting
- Underfitting
- Validation instability
- Model convergence
- Appropriate checkpoint selection

Figure 7: Training and validation loss

Figure 8: Training and validation accuracy



20. Baseline Model Results

The baseline model selected epoch 17 as the best checkpoint.

20.1 Validation Result

Best epoch: 17

Best validation loss: 0.3634

20.2 Test Results

Test loss: 0.4802

Test accuracy: 0.8137

Macro precision: 0.8034

Macro recall: 0.8228

Macro F1-score: 0.8064

Weighted F1-score: 0.8043

The baseline CNN correctly classified approximately 81.37% of the unseen test images.

The macro F1-score of 80.64% indicated reasonably balanced overall performance across the four categories.

21. Baseline Class-Wise Performance

21.1 Glioma

Precision: 0.8509

Recall: 0.7761

F1-score: 0.8118

The model achieved good precision for glioma. However, recall was lower, indicating that some actual glioma images were predicted as other classes.

21.2 Meningioma

Precision: 0.7006

Recall: 0.5367

F1-score: 0.6078

Meningioma was the weakest category.

The recall value indicated that the model correctly identified only approximately 53.67% of actual meningioma images.

This category requires further model improvement and error analysis.

21.3 No Tumor

Precision: 0.8126

Recall: 0.9923

F1-score: 0.8935

The model identified almost all actual No Tumor images.

However, precision was lower than recall, suggesting that some tumor images were incorrectly assigned to the No Tumor category.

21.4 Pituitary

Precision: 0.8494

Recall: 0.9860

F1-score: 0.9126

Pituitary was the strongest category.

The model correctly detected approximately 98.60% of actual pituitary images.

Figure : Classification report

	precision	recall	f1-score	support
glioma	0.9201	0.6683	0.7743	603.0000
meningioma	0.6438	0.7752	0.7034	436.0000
notumor	0.8819	0.9794	0.9281	389.0000
pituitary	0.9069	0.9767	0.9405	429.0000
accuracy	0.8298	0.8298	0.8298	0.8298
macro avg	0.8382	0.8499	0.8366	1857.0000
weighted avg	0.8442	0.8298	0.8283	1857.0000

Figure : Confusion matrix

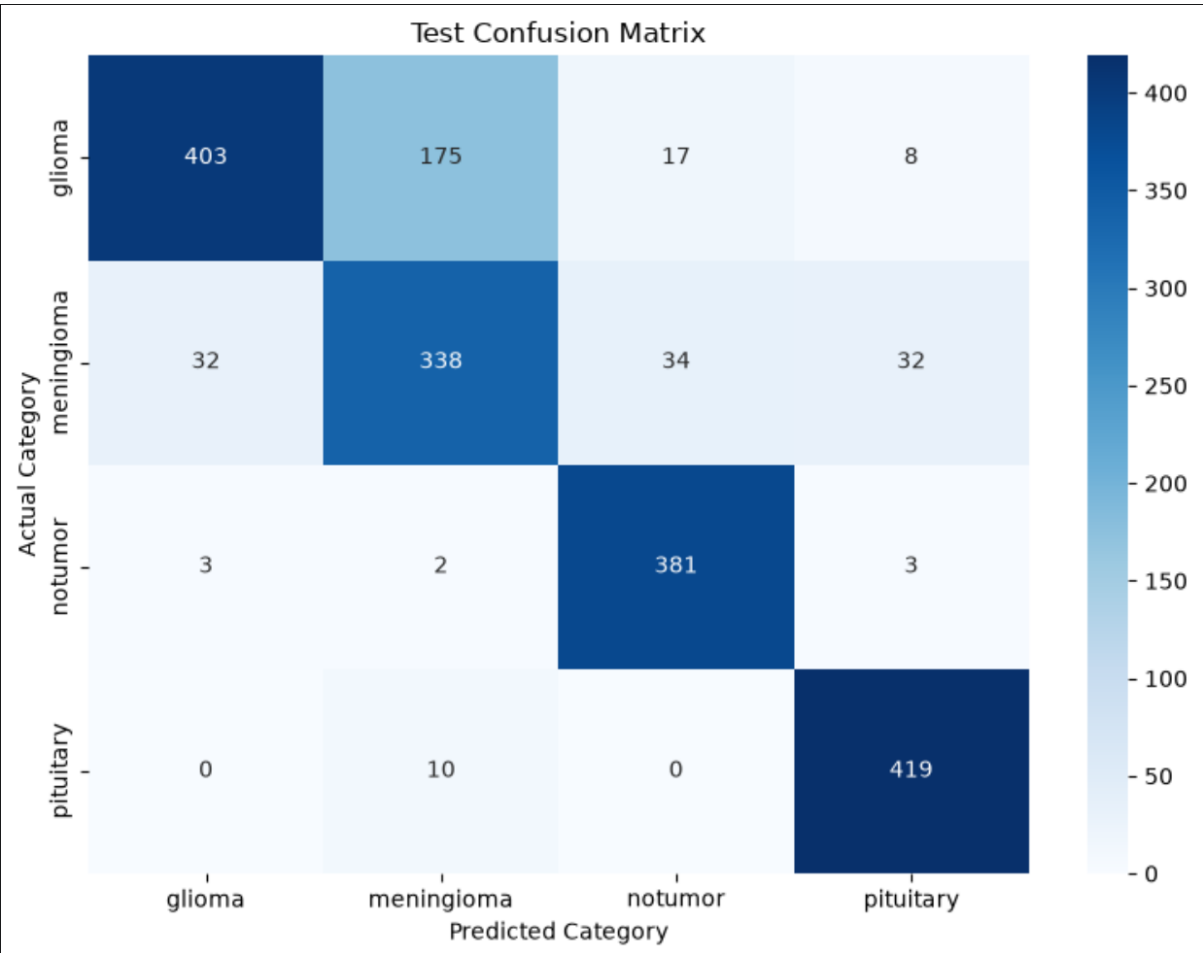
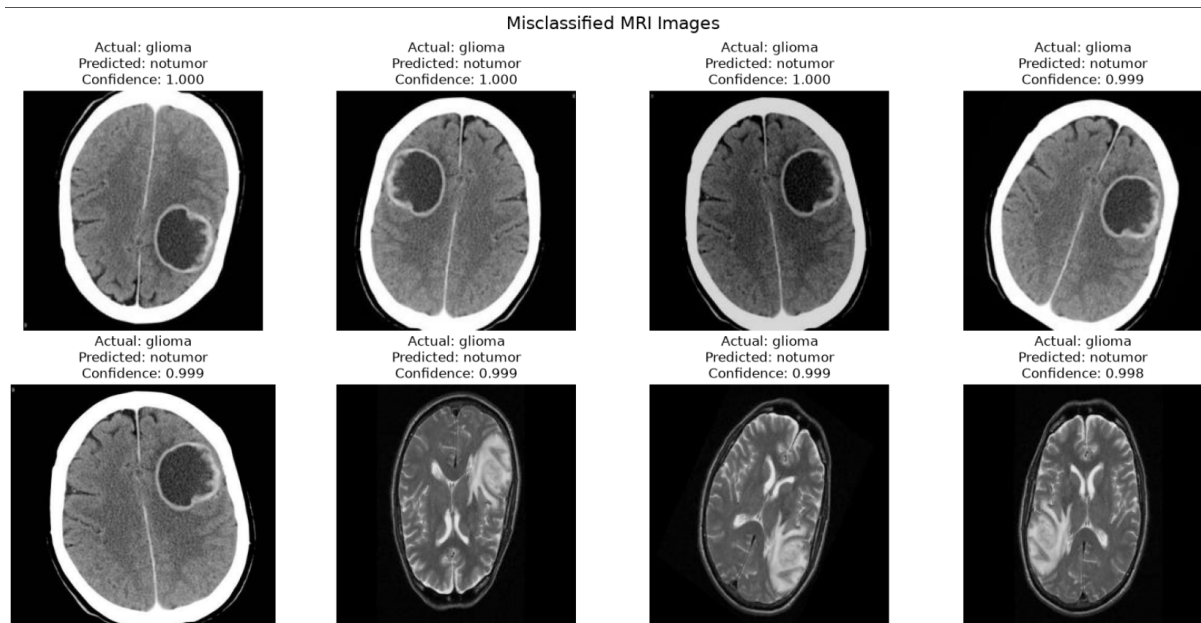


Figure : Misclassified MRI images



22. Hyperparameter Tuning

A focused hyperparameter search was performed.

The parameters evaluated were:

- Learning rate
- Weight decay
- Dropout

A full exhaustive grid search was not used because each complete training run required substantial local computation time.

Each HPT trial:

- Created a newly initialized CNN
- Used the training subset
- Used the validation subset for selection
- Did not use the test set during model selection

23. Initial HPT Experiment

The initial tuning experiment used:

Maximum epochs: 10

Patience: 3

The initial tuned model produced:

Test loss: 0.6523

Test accuracy: 0.7275

Macro precision: 0.7533

Macro recall: 0.7474

Macro F1-score: 0.7407

Weighted F1-score: 0.7307

This model performed worse than the baseline.

The main reason was that the HPT trials were limited to 10 epochs, while the baseline model reached its best checkpoint at epoch 17.

The lower learning-rate configurations also needed additional training time.

24. Improved HPT Experiment

The improved HPT experiment used:

Maximum epochs: 20

Patience: 5

Selection metric: Validation macro F1

24.1 Trial 1

Learning rate: 0.0010

Weight decay: 0.0001

Dropout: 0.40

Best epoch: 20

Validation loss: 0.2545

Validation accuracy: 0.9110

Validation macro F1: 0.9107

24.2 Trial 2

Learning rate: 0.0007

Weight decay: 0.0001

Dropout: 0.40

Best epoch: 14

Validation loss: 0.3310

Validation accuracy: 0.8824

Validation macro F1: 0.8804

24.3 Trial 3

Learning rate: 0.0010

Weight decay: 0.0005

Dropout: 0.45

Best epoch: 20

Validation loss: 0.3043

Validation accuracy: 0.8931

Validation macro F1: 0.8923

25. Selected Hyperparameter Configuration

Trial 1 was selected because it achieved:

- Lowest validation loss
- Highest validation accuracy
- Highest validation macro F1

The selected configuration was:

Learning rate: 0.001

Weight decay: 0.0001

Dropout: 0.40

Best epoch: 20

Validation loss: 0.2545

Validation accuracy: 0.9110

Validation macro F1: 0.9107

These are validation results and should not be represented as final test performance.

Figure 13: Improved HPT result comparison

	trial	learning_rate	weight_decay	dropout	best_epoch	validation_loss	validation_accuracy	validation_macro_f1
0	1	0.0010	0.0001	0.40	20	0.2545	0.9110	0.9107
1	3	0.0010	0.0005	0.45	20	0.3043	0.8931	0.8923
2	2	0.0007	0.0001	0.40	14	0.3310	0.8824	0.8804

Figure 14: Selected tuned checkpoint loaded successfully

```
Improved tuned model loaded successfully.  
Learning rate: 0.001  
Weight decay: 0.0001  
Dropout: 0.4  
Best epoch: 20  
Validation loss: 0.2545  
Validation accuracy: 0.911  
Validation macro F1: 0.9107
```

26. Final Tuned Model Test Results

Add the final test values produced by the selected improved tuned checkpoint.

```
Improved tuned test loss: 0.4106  
Improved tuned test accuracy: 0.839  
Improved tuned macro precision: 0.8355  
Improved tuned macro recall: 0.8507  
Improved tuned macro F1: 0.8412
```

Test loss: 0.4106

Test accuracy: 0.839

Macro precision: 0.8355

Macro recall: 0.8507

Macro F1-score: 0.8412

Final Tuned Class-Wise Metrics

Glioma

Precision: 0.8598

Recall: 0.7629

F1-score: 0.8084

Meningioma

Precision: 0.6983

Recall: 0.6743

F1-score: 0.6861

No Tumor

Precision: 0.8843

Recall: 0.9820

F1-score: 0.9306

Pituitary

Precision: 0.8998

Recall: 0.9837

F1-score: 0.9399

Figure : Final tuned-model classification report

	precision	recall	f1-score	support
glioma	0.8598	0.7629	0.8084	603.000
meningioma	0.6983	0.6743	0.6861	436.000
notumor	0.8843	0.9820	0.9306	389.000
pituitary	0.8998	0.9837	0.9399	429.000
accuracy	0.8390	0.8390	0.8390	0.839
macro avg	0.8355	0.8507	0.8412	1857.000
weighted avg	0.8363	0.8390	0.8357	1857.000

27. Final Model Selection

The final model was selected after considering:

- Validation macro F1
- Test loss
- Test accuracy
- Test macro F1
- Weighted F1
- Per-class recall
- Meningioma recall
- Confusion matrix
- Generalization performance

28. Single-Image Prediction

A reusable prediction function was implemented.

The prediction workflow performs the following steps:

1. Accept an image path or PIL image.
2. Verify that the image exists.
3. Convert the image to RGB.
4. Resize and normalize the image.
5. Add a batch dimension.
6. Move the tensor to the selected device.

7. Run inference mode.
8. Apply Softmax to the logits.
9. Select the class with the highest probability.
10. Return the confidence and all class probabilities.
11. Determine whether manual review is recommended.

Example Prediction

Image path:

../data/Epic and CSCR hospital Dataset/test/glioma/1353.jpg

Predicted class: glioma

Confidence: 0.9709

Manual review recommended: False

Probabilities: Glioma: 0.9709

No Tumor: 0.0290

Meningioma: approximately 0.0000

Pituitary: approximately 0.0000

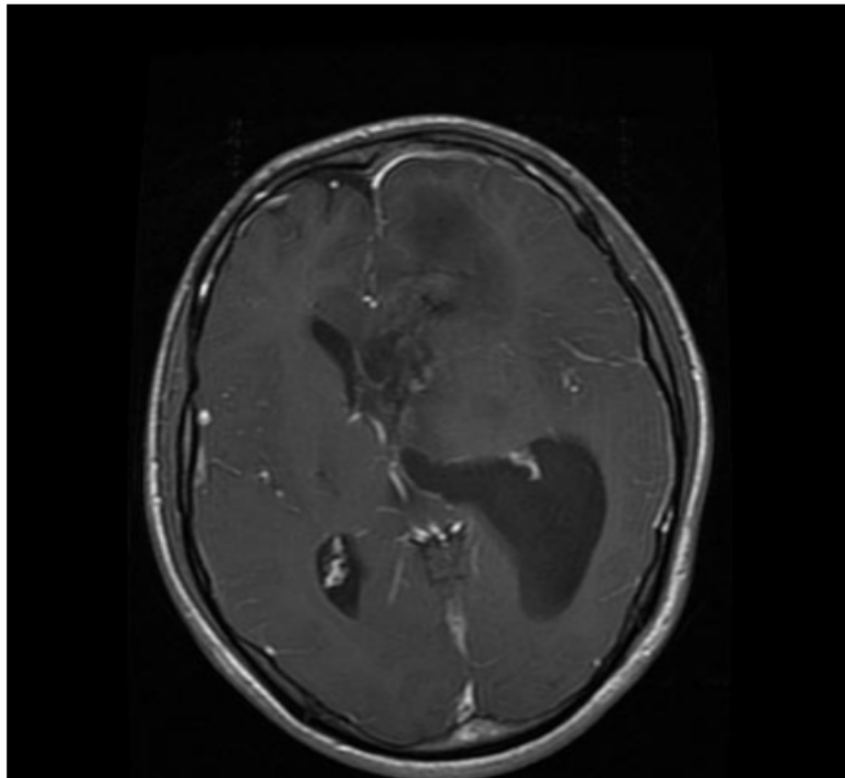
Figure 17: Single-image prediction

```
... Image path: ../data/Epic and CSCR hospital Dataset/test/glioma/1353.jpg
Predicted class: glioma
Confidence: 0.9989
Manual review recommended: False
```

```
Class probabilities:
glioma: 0.998873
notumor: 0.001092
pituitary: 0.000020
meningioma: 0.000015
```

```
...
```

Prediction: glioma

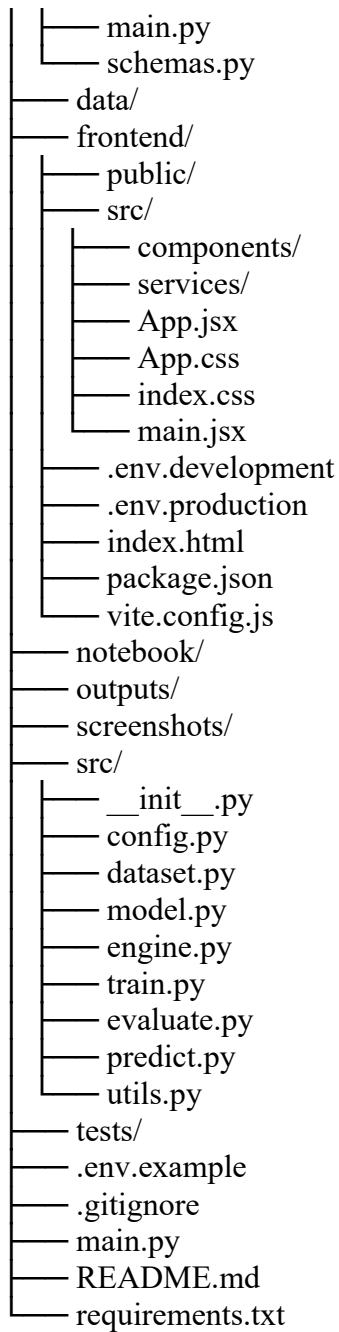


29. Production-Style Project Structure

The notebook implementation was converted into modular production-style code.

BrainTumorMRIClassification/

```
|— api/
|— __init__.py
```



30. Production Module Responsibilities

30.1 Configuration Module

`src/config.py` contains:

- Project paths

- Dataset paths
- Output paths
- Model paths
- Training hyperparameters
- Upload limits
- Supported image types
- Application name
- Application version
- CORS origins
- Environment configuration

30.2 Dataset Module

`src/dataset.py` contains:

- Image indexing
- Dataset validation
- Stratified splitting
- Training transforms
- Evaluation transforms
- Custom Dataset class

30.3 Model Module

`src/model.py` contains the custom CNN architecture.

30.4 Engine Module

`src/engine.py` contains:

- One-epoch training
- Model evaluation
- Accuracy calculation
- Macro metrics
- Weighted metrics
- Probability collection

30.5 Training Module

`src/train.py` contains:

- Dataset loading
- Split creation
- DataLoader creation

- Model initialization
- Class-weight calculation
- Loss and optimizer configuration
- Learning-rate scheduler
- Training loop
- Validation
- Early stopping
- Checkpoint saving
- Training-history export

30.6 Evaluation Module

`src/evaluate.py` contains:

- Checkpoint loading
- Test evaluation
- Classification report generation
- Confusion-matrix generation
- Prediction CSV export

30.7 Prediction Module

`src/predict.py` contains:

- Secure checkpoint loading
- Model restoration
- Metadata retrieval
- PIL-image inference
- File-path inference
- Probability generation
- Confidence-threshold logic

30.8 Utility Module

`src/utils.py` contains:

- Random-seed setup
- Device selection
- Path validation
- Device cache clearing

31. Local FastAPI Backend

A FastAPI backend was developed to serve the model.

The model was loaded once during application startup using the FastAPI lifespan.

This prevented the model from being reloaded for every request.

API Endpoints

GET /

GET /health

GET /model-info

POST /predict

31.1 Root Endpoint

The root endpoint returns:

- Application name
- Version
- Status
- Documentation path
- Health endpoint
- Disclaimer

31.2 Health Endpoint

The health endpoint returns:

- API status
- Model-loaded status
- Inference device
- Environment

31.3 Model Information Endpoint

The model-information endpoint returns:

- Model name
- Model version
- Framework
- Class list
- Input image size
- Device
- Best epoch
- Validation metrics
- Clinical-use status

31.4 Prediction Endpoint

The prediction endpoint accepts:

Image file

Confidence threshold

It returns:

- Filename
- Predicted class
- Confidence
- Manual-review status
- Confidence threshold
- Four class probabilities
- Disclaimer

32. Backend Input Validation

The API validates:

- Model availability
- Confidence-threshold range
- File content type
- File size
- Empty uploads
- Image integrity
- Unsupported file formats

The maximum upload size is: 10 MB

Supported request types are: image/jpeg

image/png

image/webp

Uploaded MRI images are processed in memory.

The backend does not write uploaded images to disk.

33. Local React Frontend

A React frontend was developed using Vite.

The frontend includes:

- Application title
- API connection status
- Image selection
- Drag-and-drop upload
- Image preview
- Filename and file-size display
- File-type validation
- File-size validation
- Prediction loading indicator
- Prediction result
- Confidence bar
- Class probabilities
- Manual-review warning
- Reset button
- Responsive layout
- Educational disclaimer

33.1 Local Development API Configuration

The local frontend used:

VITE_API_URL=http://127.0.0.1:8000

This value was stored in:

frontend/.env.development

33.2 Production Configuration

The AWS backend URL was stored in:

frontend/.env.production

Example: VITE_API_URL=http://[EC2_ELASTIC_IP]

Vite embedded the production URL during:

npm run build

34. Complete Local Workflow

User opens React frontend

↓

Frontend checks FastAPI health

↓

User chooses an MRI file

↓

Frontend validates type and size

↓

Frontend displays image preview

↓

User selects Run Prediction

↓

Frontend submits multipart request

↓

FastAPI validates uploaded image

↓

Pillow converts image to RGB

↓

PyTorch transformations are applied

↓

CNN performs inference

↓

Softmax probabilities are calculated

↓

FastAPI returns JSON

↓

React displays the prediction

Figure : Local frontend home screen

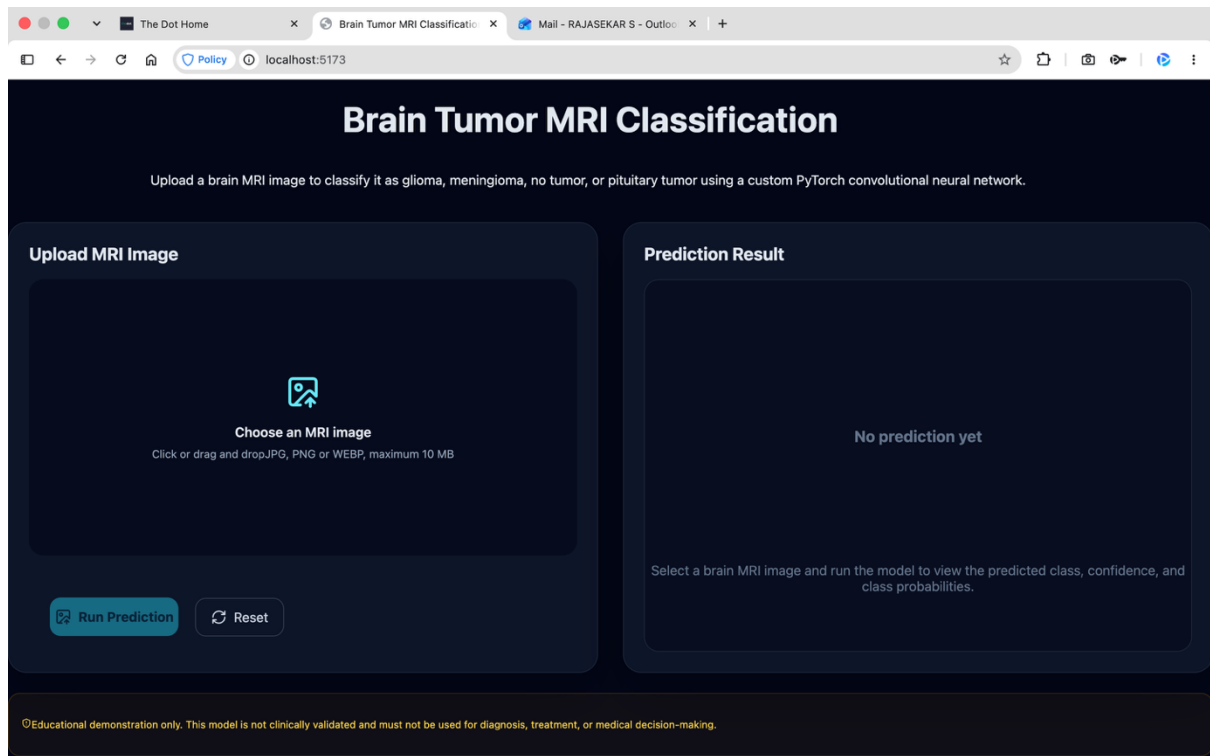


Figure 19: Local image preview

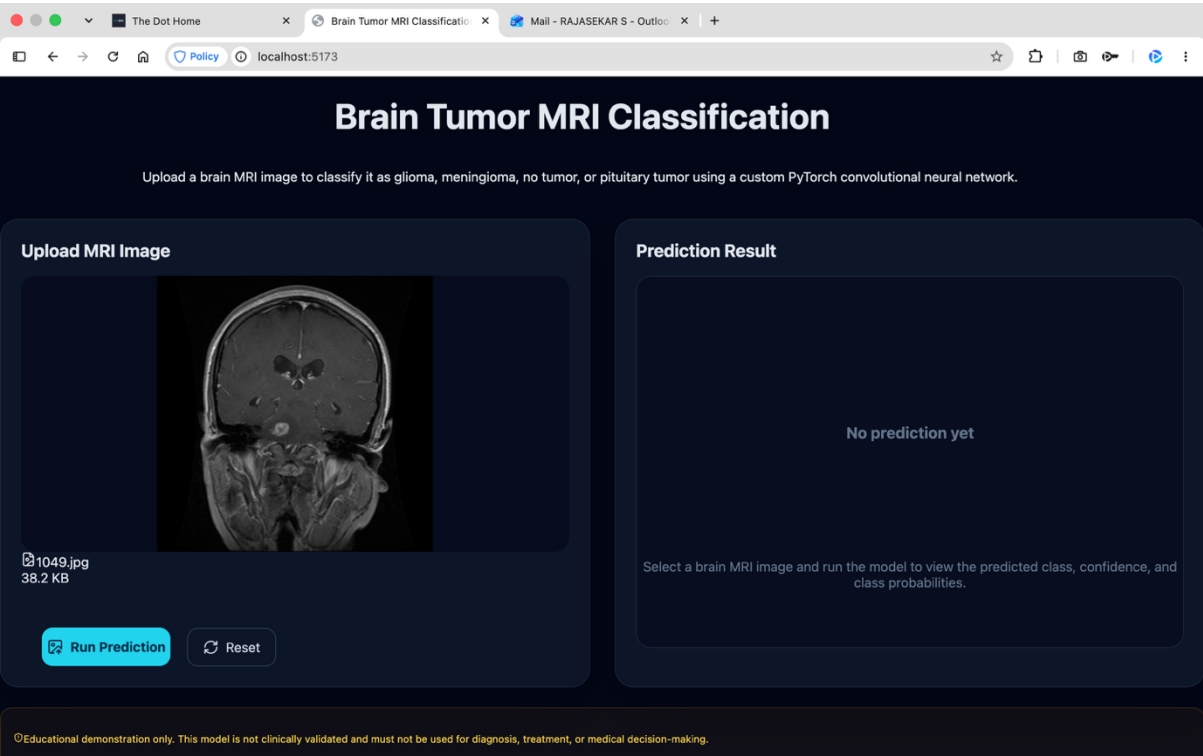


Figure 20: Successful local frontend prediction

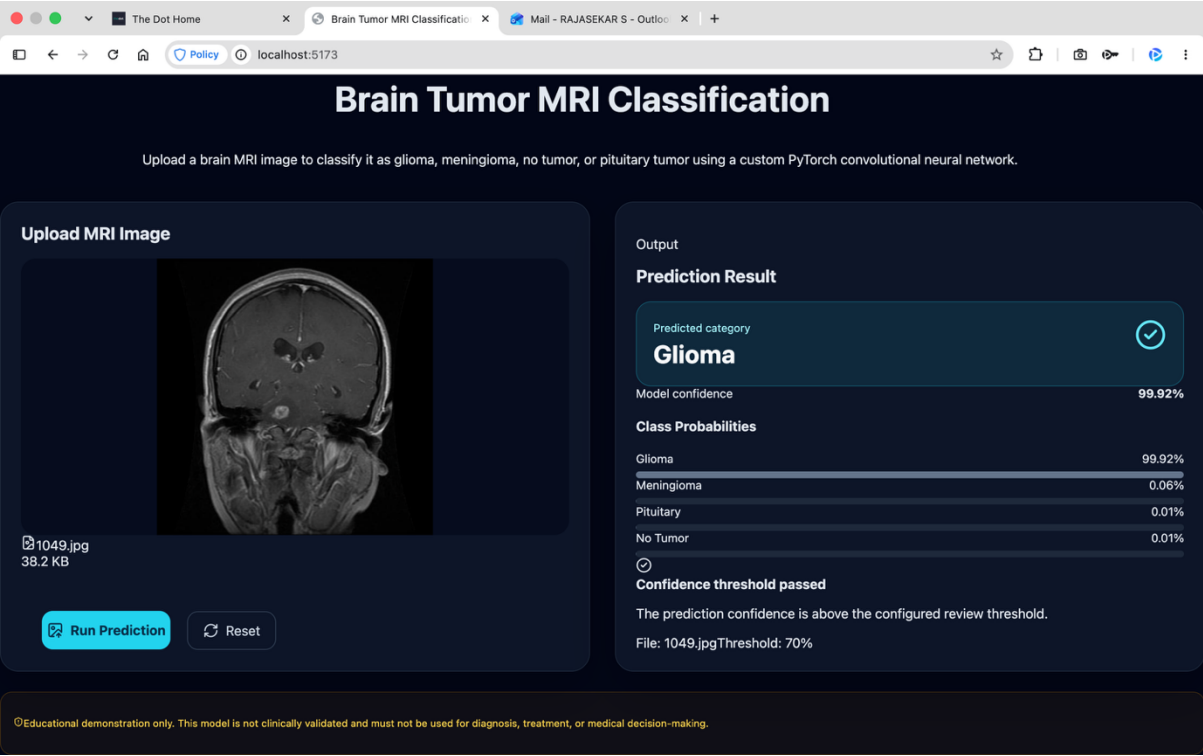
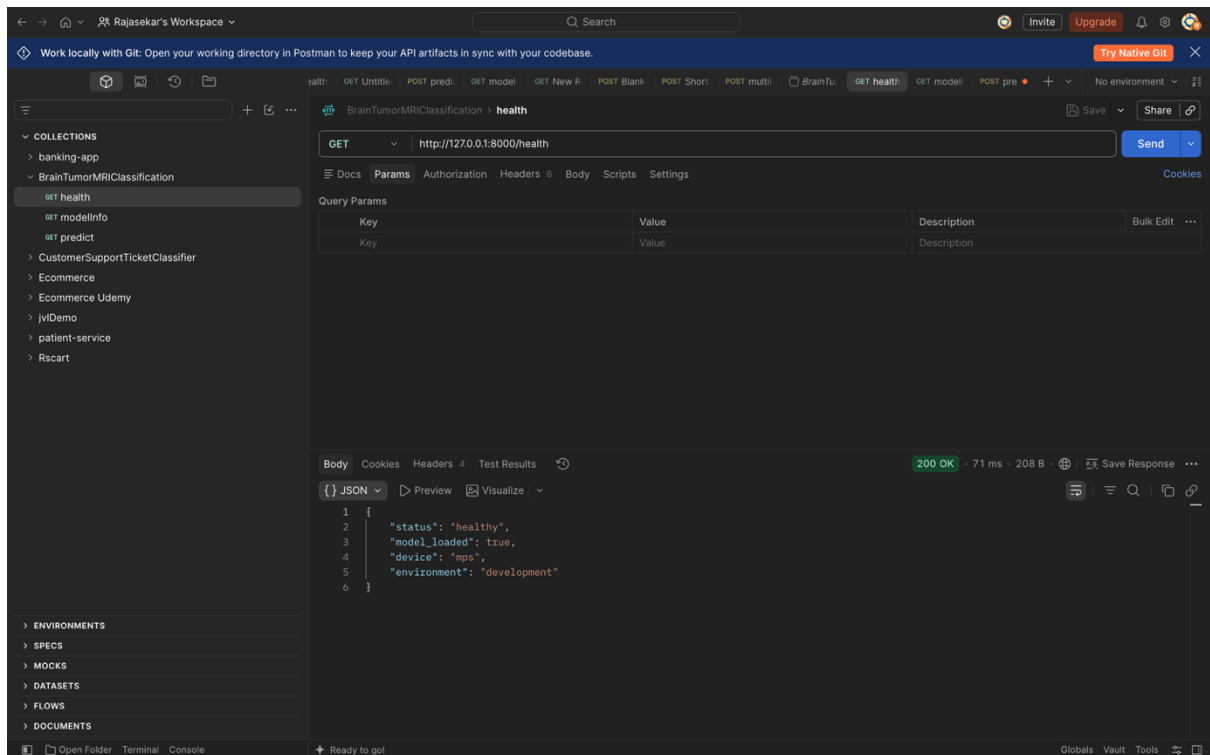


Figure 21: Local Swagger or Postman prediction

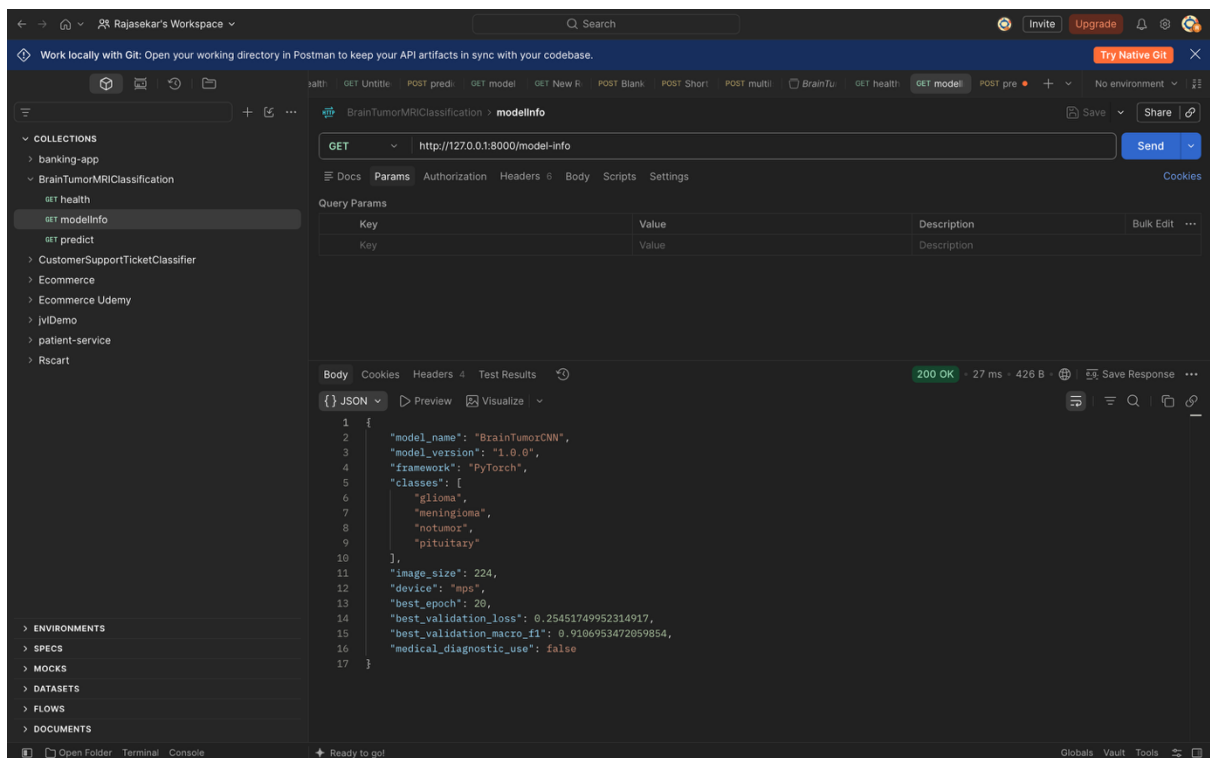
/health



A screenshot of the Postman application interface. The left sidebar shows a collection named 'BrainTumorMRIClassification' with a sub-collection 'health' selected. The main panel displays a GET request to 'http://127.0.0.1:8000/health'. The response is a 200 OK status with a response time of 71 ms and a body of 208 B. The response body is shown in JSON format:

```
1 {
2   "status": "healthy",
3   "model_loaded": true,
4   "device": "mps",
5   "environment": "development"
6 }
```

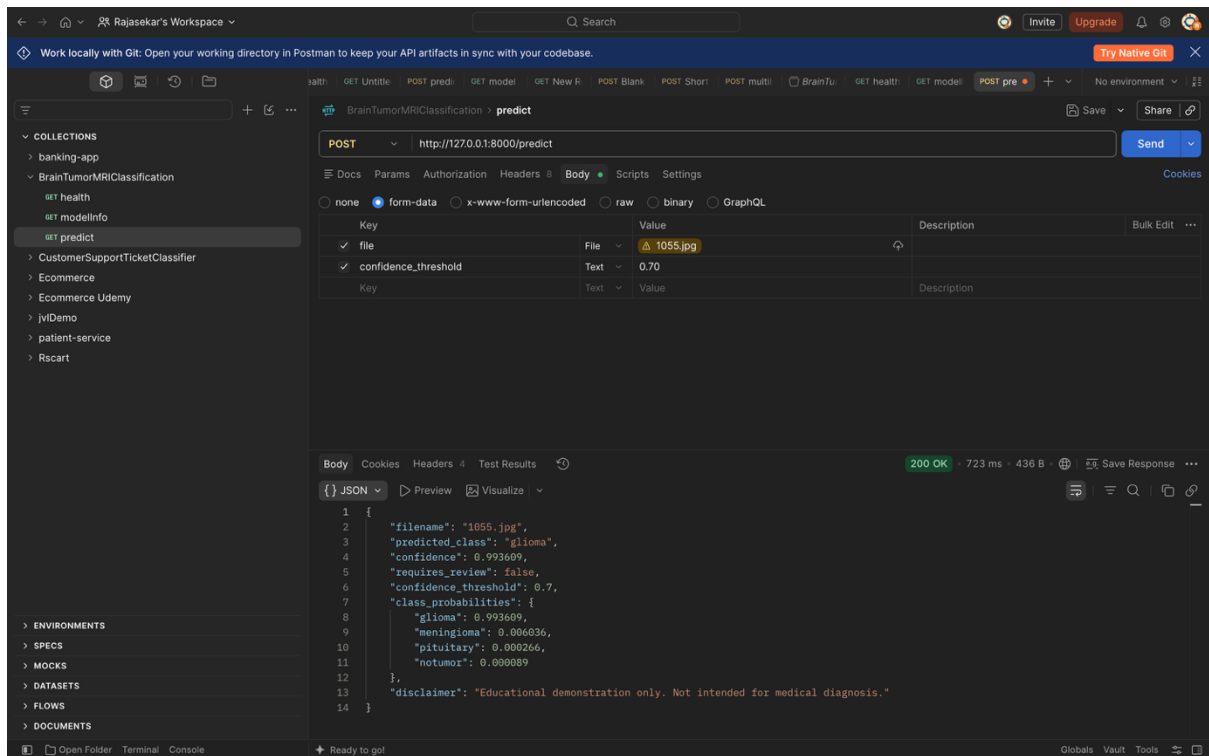
/model-info



A screenshot of the Postman application interface. The left sidebar shows a collection named 'BrainTumorMRIClassification' with a sub-collection 'modelInfo' selected. The main panel displays a GET request to 'http://127.0.0.1:8000/model-info'. The response is a 200 OK status with a response time of 27 ms and a body of 426 B. The response body is shown in JSON format:

```
1 {
2   "model_name": "BrainTumorCNN",
3   "model_version": "1.0.0",
4   "framework": "PyTorch",
5   "classes": [
6     "glioma",
7     "meningioma",
8     "notumor",
9     "pituitary"
10  ],
11   "image_size": 224,
12   "device": "mps",
13   "best_epoch": 20,
14   "best_validation_loss": 0.25451749952314917,
15   "best_validation_macro_f1": 0.9106953472059854,
16   "medical_diagnostic_use": false
17 }
```

/predict



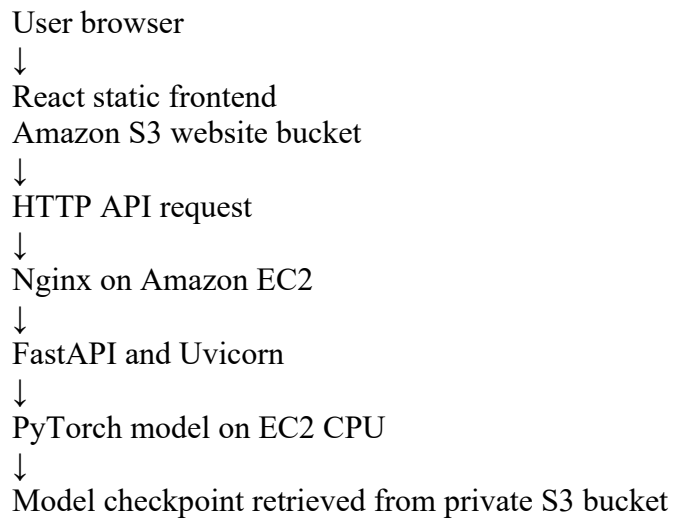
35. Local Testing

The following local tests were completed:

- FastAPI startup
- Root endpoint
- Health endpoint
- Model information endpoint
- Valid image prediction
- Invalid file-type rejection
- Invalid confidence-threshold rejection
- Oversized image rejection
- Corrupted-image rejection
- Frontend API connection check
- Image preview
- Prediction display
- Class-probability display
- Reset functionality
- API disconnected state
- React production build

36. AWS Deployment Architecture

The completed application was deployed using the following architecture:



AWS Services Used

- Amazon S3 frontend bucket
- Amazon S3 private model-artifact bucket
- Amazon EC2
- AWS IAM
- EC2 security group
- Elastic IP
- Nginx
- systemd

37. S3 React Static Website Deployment

The React production build was generated using:

```
npm run build
```

This created:

```
frontend/dist/  
├── index.html  
└── assets/
```

The generated `index.html` and asset files were uploaded to the root of the S3 frontend bucket.

The final S3 object structure was:

```
index.html
assets/
├── generated JavaScript file
├── generated CSS file
└── additional generated assets
```

Static website hosting was enabled with:

Index document: index.html

Error document: index.html

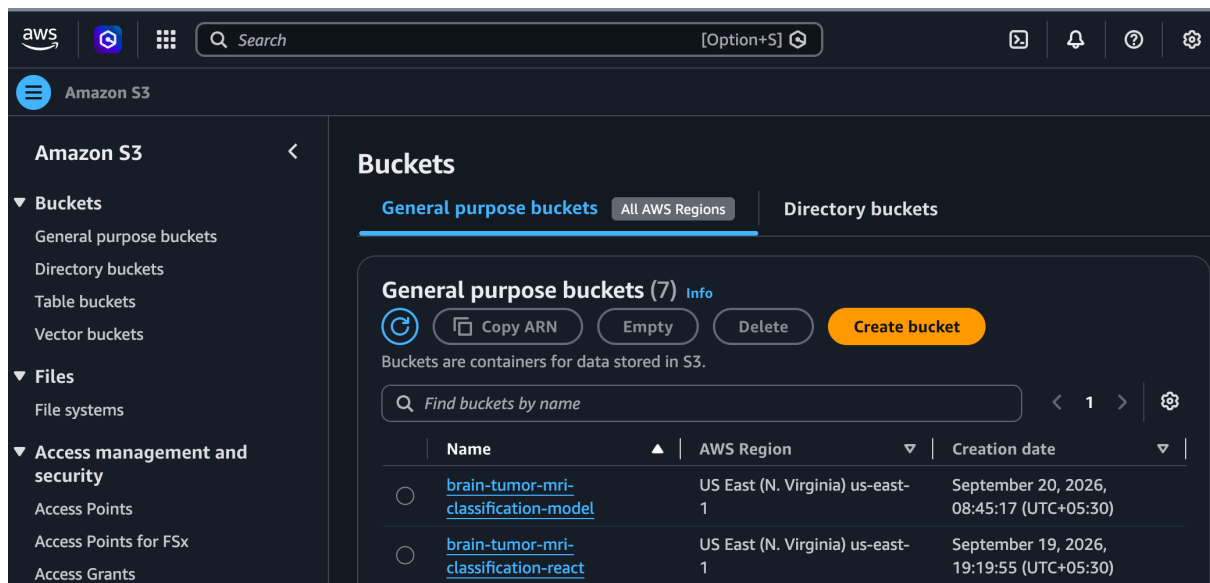
The error document was set to `index.html` to support the React single-page application.

37.1 Frontend Bucket

Bucket Name:

brain-tumor-mri-classification-react

Website Endpoint:



37.2 Public Read Configuration

The frontend bucket was configured for public object read access.

Only the generated static frontend files were stored in this bucket.

The following sensitive or private items were not uploaded:

- MRI dataset
- Model checkpoint
- Environment files
- AWS credentials
- Private documents
- Access keys

Figure 25: S3 frontend bucket objects

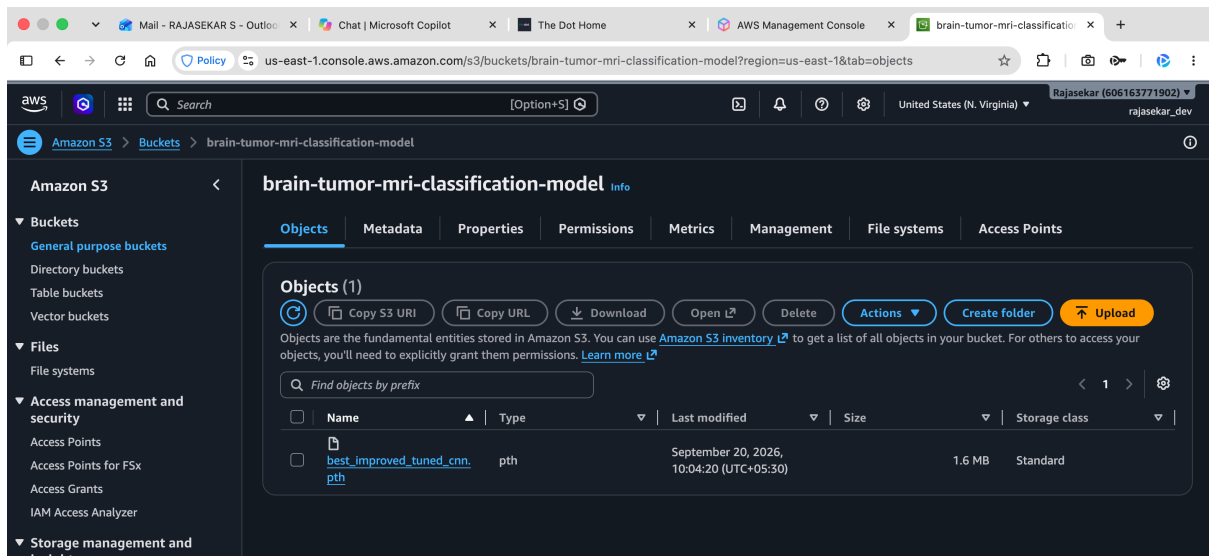
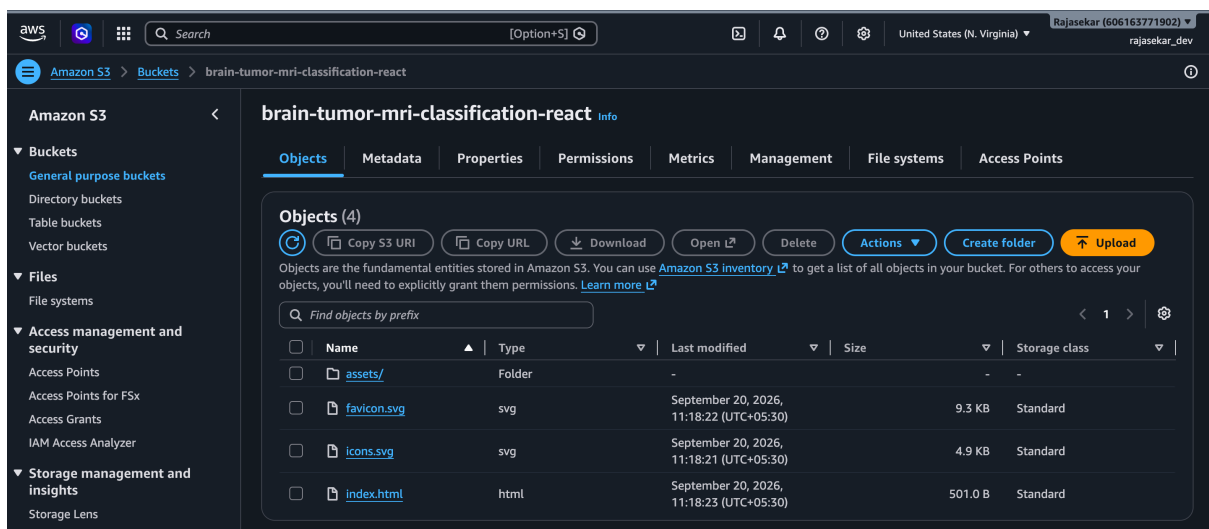


Figure 26: S3 static website configuration



38. Private S3 Model Artifact Storage

The trained model was stored in a separate private S3 bucket.

The public React frontend bucket was not used for model storage.

38.1 Model Bucket Structure

Private model bucket

- └─ models/
- └─ v1/
- └─ selected model checkpoint

38.2 Model Artifact

S3 Model Location:

s3://brain-tumor-mri-classification-model/best_improved_tuned_cnn.pth

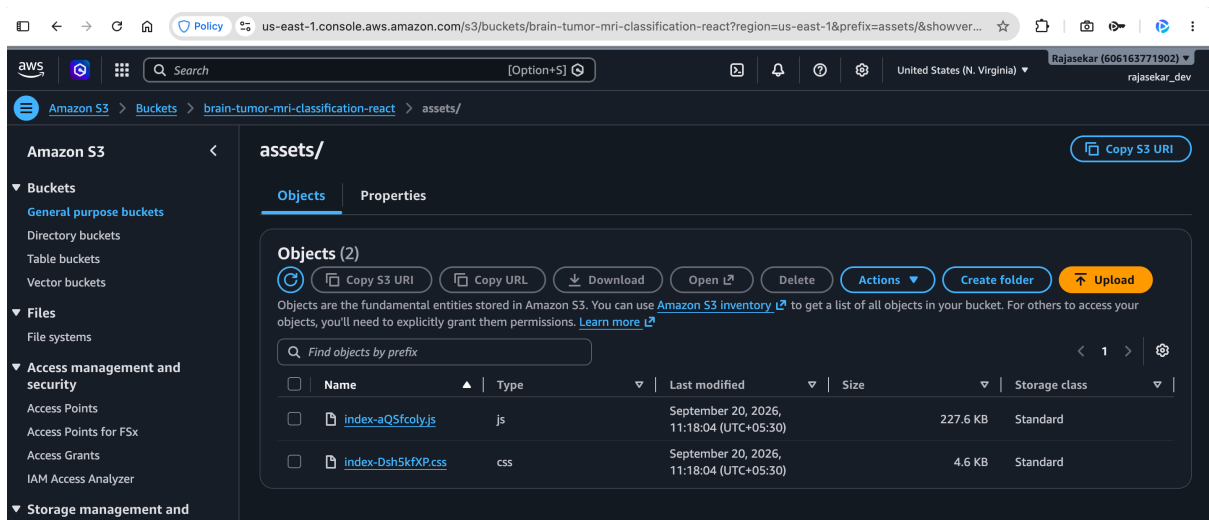
38.3 Security Configuration

The model bucket was configured with:

- Block Public Access enabled
- Static website hosting disabled
- Bucket-owner-enforced object ownership
- Server-side encryption
- Versioning, if enabled
- IAM-controlled access

The model was not made publicly readable.

Figure 28: Private model-artifact bucket



39. EC2 IAM Role

An IAM role was created for the EC2 instance.

The role granted only the permission required to download the production model checkpoint.

The main permission was:

s3:GetObject

The permission was restricted to the production-model object path.

No AWS access key or secret key was stored on EC2.

The EC2 instance used temporary IAM role credentials through the instance profile.

40. EC2 Backend Deployment

An Ubuntu EC2 instance was created for the FastAPI backend.

EC2 Configuration

Instance name: brain-tumor-api

Operating system: Ubuntu

Instance type: m7i-flex.large

Storage: GP3 SSD

Inference device: CPU

The following packages were installed:

- Python
- pip
- Python virtual environment
- Git
- Nginx
- Curl
- PyTorch CPU package
- FastAPI dependencies
- Boto3

The project repository was cloned from GitHub.

A Python virtual environment was created and the requirements were installed.

The selected model was downloaded from the private S3 bucket using Boto3 and the EC2 IAM role.

41. EC2 Security Group

The EC2 security group allowed:

SSH

Port: 22

Source: My IP

HTTP

Port: 80

Source: 0.0.0.0/0

The FastAPI Uvicorn port was not exposed permanently to the public internet.

Nginx handled public traffic on port 80 and forwarded requests internally to:

127.0.0.1:8000

42. Elastic IP

An Elastic IP was associated with the EC2 instance.

This provided a stable public address for the backend.

Backend URL:

`http://[EC2_ELASTIC_IP]`

Health URL:

`http://[EC2_ELASTIC_IP]/health`

Model Information URL:

`http://[EC2_ELASTIC_IP]/model-info`

Swagger URL:

`http://[EC2_ELASTIC_IP]/docs`

43. FastAPI Service Management

A systemd service was created to run the FastAPI backend continuously.

The service:

- Starts after the network becomes available
- Uses the project virtual environment
- Loads environment variables
- Runs Uvicorn on `127.0.0.1:8000`
- Restarts automatically after failure
- Starts automatically after EC2 reboot

Systemd Service Name:

brain-tumor-api

Service status was verified as:

active (running)

44. Nginx Reverse Proxy

Nginx was configured as the public web server.

It received HTTP traffic on port 80 and forwarded it to FastAPI.

Browser request

↓

Nginx port 80

↓

FastAPI 127.0.0.1:8000

Nginx was configured with:

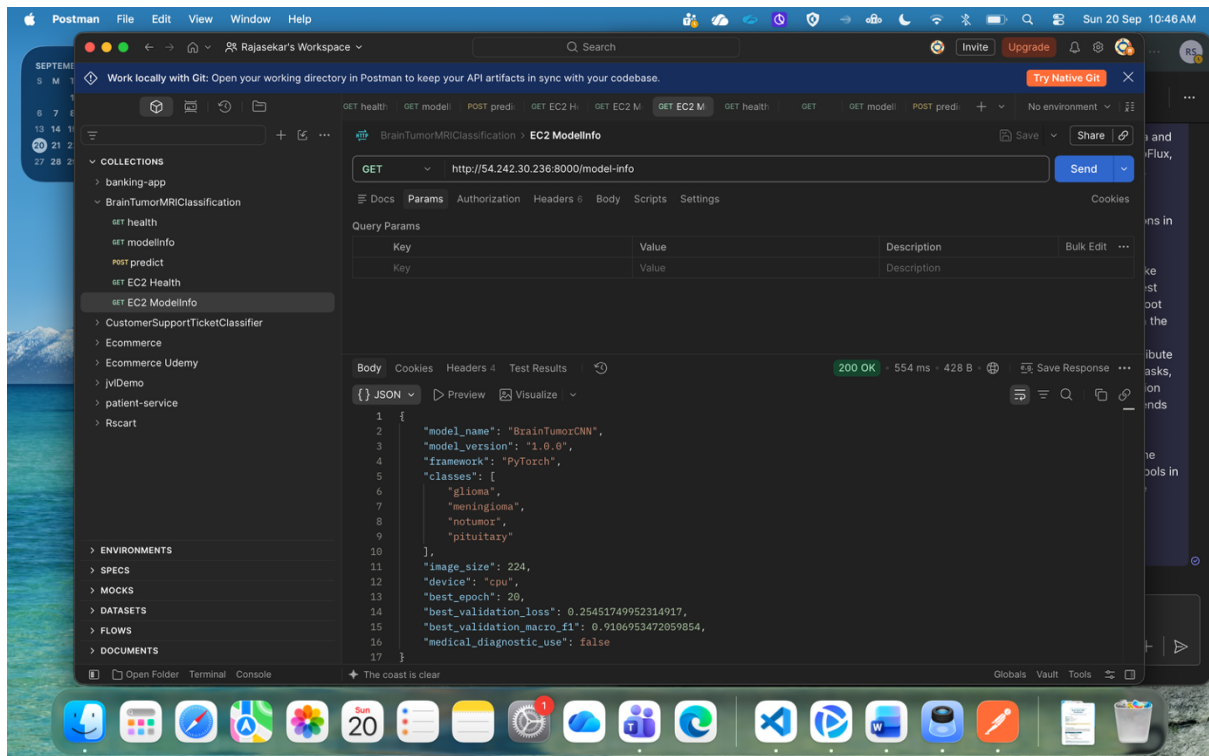
- Reverse proxy
- Client upload size limit
- Forwarded request headers
- Connection timeout
- Send timeout
- Read timeout

This prevented the internal FastAPI port from being directly exposed to the internet.

45. AWS Backend Validation

The following AWS endpoints were tested:

Health Endpoint



GET <http://54.242.30.238:8000/health>

Expected response:

Status: healthy

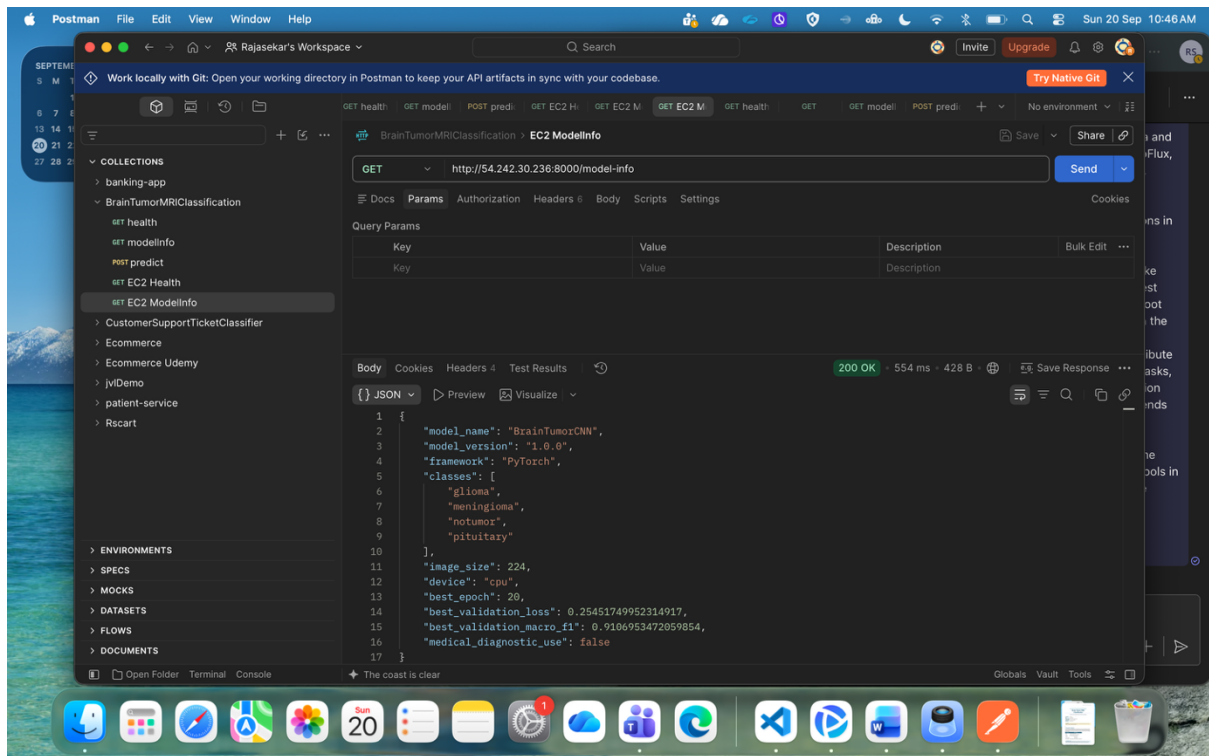
Model loaded: true

Device: cpu

Environment: production

Model Information Endpoint

GET [http:// 54.242.30.238:8000/model-info](http://54.242.30.238:8000/model-info)

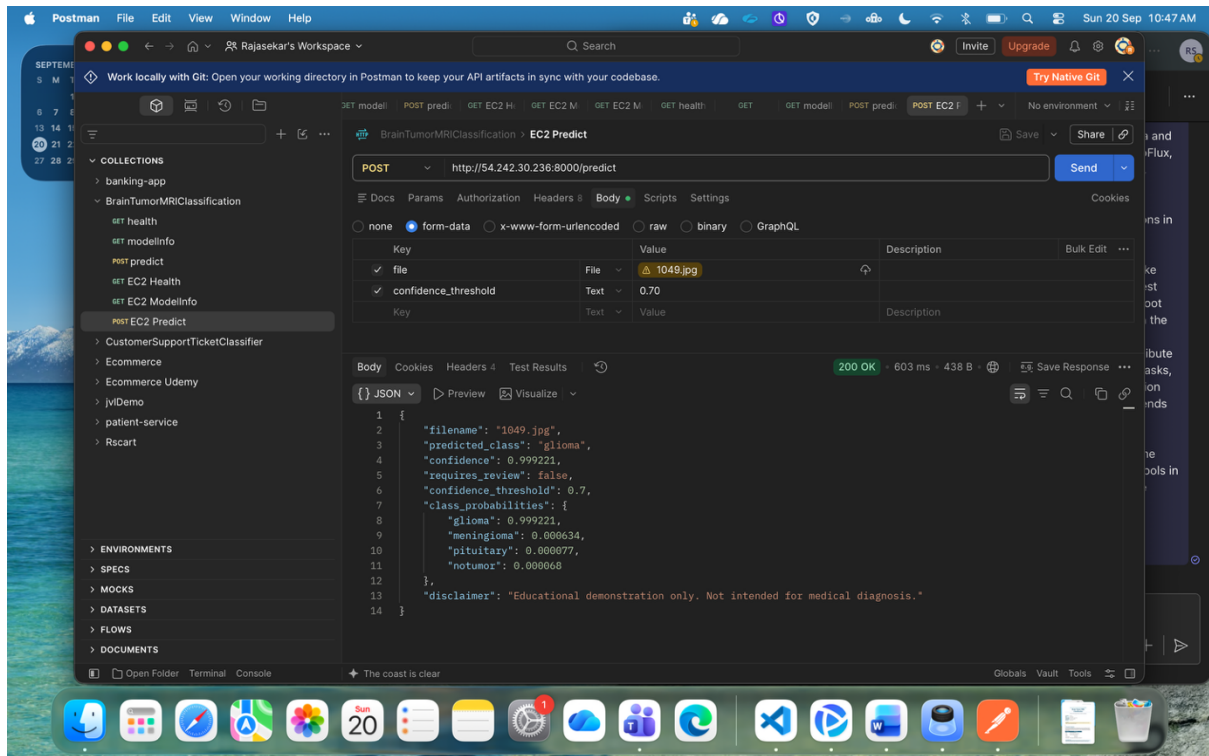


The response confirmed:

- Model name
- Model version
- Classes
- Input image size
- CPU device
- Validation metadata

Prediction Endpoint

POST [http:// 54.242.30.238:8000/ predict](http://54.242.30.238:8000/predict)



The prediction response contained:

- Filename
- Predicted class
- Confidence
- Review status
- All four class probabilities
- Disclaimer

46. CORS Configuration

The frontend and backend used different browser origins.

The backend CORS configuration included:

- Local Vite development origin
- Local Vite preview origin
- S3 static website origin

Example origins included:

`http://localhost:5173`

`http://127.0.0.1:5173`

`http://localhost:4173`

`http://127.0.0.1:4173`

`http://[S3_WEBSITE_ENDPOINT]`

Trailing slashes were not included.

The backend service was restarted after updating CORS configuration.

47. Production React Build

The React application used different environment files.

Development

`frontend/.env.development`

`VITE_API_URL=http://127.0.0.1:8000`

Production

`frontend/.env.production`

`VITE_API_URL=http://[EC2_ELASTIC_IP]`

`npm run dev` used the development environment.

`npm run build` used the production environment.

The application was rebuilt after adding the EC2 backend URL.

The newly generated `index.html` and generated asset files were uploaded to S3.

Old generated asset files were removed to avoid mismatched Vite hashes.

48. Complete AWS Workflow

User opens S3 website
↓
React production application loads
↓
Frontend calls EC2 health endpoint
↓
API status shows connected
↓
User selects MRI image
↓
React validates and previews image
↓
Frontend sends multipart request
↓
EC2 Nginx receives request
↓
Nginx forwards request to FastAPI
↓
FastAPI preprocesses image
↓
PyTorch CNN performs CPU inference
↓
FastAPI returns JSON result
↓
React displays prediction and probabilities

49. Local and AWS Environment Comparison

Local Environment

Frontend: Vite development server

Backend: Local FastAPI

Model device: Apple M1 MPS

API URL: `http://127.0.0.1:8000`

AWS Environment

Frontend: Amazon S3 static website

Backend: Amazon EC2

Reverse proxy: Nginx

Service management: systemd

Model device: CPU

Model storage: Private Amazon S3 bucket

API URL: EC2 Elastic IP

The same preprocessing and model architecture were used in both environments.

50. Challenges and Resolutions

50.1 macOS Multiprocessing Error

Issue: The custom Dataset could not be loaded by DataLoader worker processes.

Resolution: DataLoader workers were set to zero.

50.2 Model Initialization Error

Issue: The optimizer was configured before the model existed.

Resolution: The model was initialized and moved to the device before configuring the optimizer.

50.3 Initial HPT Underperformance

Issue: The first tuned model performed worse than the baseline.

Cause: HPT was limited to 10 epochs, while the baseline best epoch was 17.

Resolution: Maximum epochs were increased to 20, patience was increased to five, and validation macro F1 was used for selection.

50.4 Checkpoint Loading Error

Issue: PyTorch restricted NumPy metadata during checkpoint loading.

Resolution: Metadata was converted to native Python values. Unrestricted loading was used only for the trusted locally generated checkpoint.

50.5 FastAPI Import Errors

Issue: Configuration names and prediction functions were inconsistent between modules.

Resolution: Imports and function signatures were synchronized across `config.py`, `predict.py`, and `api/main.py`.

50.6 React Image Preview Error

Issue: Selecting an image produced a blank page.

Cause: The preview URL was rendered as an object rather than an image source.

Resolution: The preview was rendered as an image element using the preview URL as its source.

50.7 Vite Index File Error

Issue: The source `index.html` referenced an old generated JavaScript file.

Cause: A generated `dist/index.html` was copied back into the source directory.

Resolution: The source `index.html` was restored to reference `/src/main.jsx`, and Vite regenerated the production asset references.

50.8 S3 Upload Forbidden Error

Issue: S3 file uploads failed with an HTTP 403 response.

Cause: The active AWS identity did not have sufficient permission for all bucket objects.

Resolution: Bucket-level and object-level permissions were corrected. Public read access was limited to the frontend objects, while model artifacts remained private.

50.9 Local Frontend Could Not Reach EC2

Issue: EC2 health worked in the browser, but the local React frontend could not connect.

Cause: The local frontend origin was not included in the EC2 backend CORS configuration.

Resolution: Local origins were added to the FastAPI CORS list, and the backend service was restarted.

50.10 Development and Production Environment Confusion

Issue: The Vite development server continued using the local API URL.

Resolution: Environment usage was clarified:

npm run dev -> .env.development

npm run build -> .env.production

npm run preview -> generated production build

51. Security and Privacy Considerations

The project included the following controls:

- Public frontend and private model artifacts were stored separately.
- The model bucket blocked public access.
- Uploaded MRI files were processed in memory.
- Uploaded images were not persisted.

- File type and size were validated.
 - Corrupted images were rejected.
 - CORS origins were explicitly configured.
 - SSH access was limited to the deployment IP.
 - The internal Uvicorn port was not permanently public.
 - Nginx handled external API traffic.
 - The EC2 instance used an IAM role instead of stored access keys.
 - Environment files were excluded from source control.
 - The dataset was excluded from GitHub.
 - The model checkpoint was not placed in the public frontend bucket.
-

52. Known Limitations

The project has the following limitations:

1. The model was trained on one dataset.
 2. Performance may differ across MRI scanners and institutions.
 3. Patient-level independence could not be guaranteed if patient identifiers were unavailable.
 4. The model performs classification only.
 5. The model does not perform tumor segmentation.
 6. The model does not identify tumor location.
 7. Meningioma was difficult for the baseline model.
 8. Softmax confidence is not medical certainty.
 9. Probabilities have not been clinically calibrated.
 10. The API does not currently include authentication.
 11. Prediction history is not stored.
 12. The deployed backend currently uses CPU inference.
 13. The S3 static website endpoint uses HTTP.
 14. The backend requires HTTPS before the frontend is moved to a secure CloudFront URL.
 15. The model has not undergone external validation.
 16. The application is not suitable for clinical use.
-

53. Future Improvements

Model Improvements

- ResNet18 transfer learning
- EfficientNet comparison
- Model ensembles

- Focal loss
- Label smoothing
- Improved augmentation
- Improved meningioma recall
- Patient-level splitting
- Probability calibration
- External validation
- Grad-CAM explainability

Backend Improvements

- HTTPS
- Authentication
- Authorization
- Rate limiting
- Structured logging
- Request identifiers
- CloudWatch integration
- Model version endpoint
- Prediction history
- Automated health monitoring

Frontend Improvements

- Grad-CAM visualization
- Model version display
- Adjustable confidence threshold
- Upload progress
- Prediction history
- Improved accessibility
- Automated frontend testing
- Secure CloudFront hosting

AWS Improvements

- CloudFront frontend distribution
- Private S3 frontend origin using Origin Access Control
- HTTPS API domain
- Application Load Balancer
- AWS Certificate Manager
- CloudWatch monitoring
- Automated deployment
- CI/CD
- Infrastructure as Code
- Auto Scaling, if required

54. GitHub Repository

Repository Name: BrainTumorMRIClassification

Repository URL: <https://github.com/Raj2k01/BrainTumorMRIClassification>

Branch: main

The repository includes:

- Notebook
- Modular source code
- FastAPI backend
- React frontend
- Test code
- Dependency files
- Environment examples
- Documentation

The repository excludes:

- MRI dataset
 - Python virtual environment
 - Node modules
 - Environment secrets
 - Temporary cache
 - Private S3 artifacts
 - AWS credentials
-

56. Final Conclusion

The Brain Tumor MRI Classification project successfully implemented a complete deep learning application from data preparation to AWS deployment.

The project included:

- Dataset validation
- Stratified data splitting
- Image preprocessing
- Data augmentation
- Custom CNN development
- MPS-accelerated training
- Weighted loss

- Learning-rate scheduling
- Early stopping
- Checkpointing
- Baseline evaluation
- Hyperparameter tuning
- Class-wise performance analysis
- Modular production-style Python code
- FastAPI model serving
- Input validation
- React frontend development
- Local end-to-end integration
- S3 static website hosting
- Private S3 model artifact storage
- EC2 backend deployment
- IAM role configuration
- Nginx reverse proxy
- systemd service management
- AWS end-to-end testing

The baseline CNN achieved:

Test accuracy: 81.37%

Macro F1-score: 80.64%

The improved HPT experiment selected a configuration with:

Validation accuracy: 91.10%

Validation macro F1: 91.07%

Validation loss: 0.2545

The completed application allows a user to upload a brain MRI image and receive:

- Predicted tumor category
- Prediction confidence
- Probabilities for all four classes
- Manual-review indicator

The project provides practical evidence of deep learning, computer vision, Python development, API engineering, frontend integration, AWS deployment, security-aware model artifact handling, and end-to-end machine learning engineering.

Although the model is not suitable for clinical use, the implementation successfully meets the educational and portfolio objectives of the project.

